EE24B092
Avisha Mathur
<u>Lab Report Assignment 2</u>

(This report includes the description inside the code itself, as it is easier to explain what I have implemented side by side. The explanation is written in black.)

1. find_temperature_extremes() - Identify hottest and coldest months on record for a city

```
def find_temperature_extremes(filename, city_name):
    """
    Find the hottest and coldest months on record for a city.

    Parameters:
    filename (str): Path to the CSV file
    city_name (str): Name of the city

    Returns:
    dict: {
        'hottest': {'date': 'YYYY-MM', 'temperature': float},
        'coldest': {'date': 'YYYY-MM', 'temperature': float}
    }

    """
```

We check if the city exists in the first place.

```
    if city_name not in get_available_cities(filename):
        raise ValueError("City not found")

    #initialise to (unrealistically) high and low values.
    ext_temp = {
        'hottest': {'date': '', 'temperature': -1000},
        'coldest': {'date': '', 'temperature': 1000}
    }
```

y_to_temp is a dictionary that maps dates to temperatures on that date.

```
    y_to_temp = get_city_temperatures(filename, city_name)
    for year, temperature in y_to_temp.items():
        #check if current temp > highest till now or < lowest till now
        if temperature > ext_temp['hottest']['temperature']:
            ext_temp['hottest']['temperature'] = temperature
            ext_temp['hottest']['date'] = year

        if temperature < ext_temp['coldest']['temperature']:
            ext_temp['coldest']['temperature'] = temperature
            ext_temp['coldest']['date'] = year
```

```
    return ext_temp
```

2. get_seasonal_averages() - Calculate seasonal temperature patterns (spring, summer, fall, winter)

```
def get_seasonal_averages(filename, city_name, season):
    """
    Calculate average temperature for a specific season across all years.
    Never mind that Chennai only has Hot, Hotter and Hottest...

    Parameters:
    filename (str): Path to the CSV file
    city_name (str): Name of the city
    season (str): 'spring', 'summer', 'fall', or 'winter'

    Returns:
    dict: {
        'city': str,
        'season': str,
        'average_temperature': float
    }

    Assume: Spring = Mar,Apr,May; Summer = Jun,Jul,Aug;
          Fall = Sep,Oct,Nov; Winter = Dec,Jan,Feb
    """
```

Check if city exists

```
    if city_name not in get_available_cities(filename):
        raise ValueError("City not found")
```

Because we are taking the seasonal average, we will sum it up, and then divide it by the number of observations at the end.

```
    n = 0 #number of times that season comes up for this city
    sea_av = {
        'city': city_name,
        'season': season,
        'average_temperature': 0
    }
```

tocheck is the list that contains all the months we need to look for, for that particular season

```
    #lists of number of months to check.
    if season == 'spring':
        tocheck = [i for i in range(3, 6)]
    elif season == 'summer':
```

```
        tocheck = [i for i in range(6, 9)]
    elif season == 'fall':
        tocheck = [i for i in range(9, 12)]
    elif season == 'winter':
        tocheck = [12, 1, 2]
```

Raise error if the season doesn't exist.

```
    else:
        #incase the season isn't one of the four specified
        raise ValueError("Invalid Season")


    with open(filename, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file)
        n = 0
        for row in reader:
            if row['City'] == city_name:
                month = row['dt'][5:7]
```

If the date falls in one of the months, add temp to the sum (if it is non-null).

```
                if int(month) in tocheck and row['AverageTemperature'] != '':
```

#if it is in the range, add to running sum

```
                    n += 1
                    sea_av['average_temperature'] +=
float(row['AverageTemperature']) #keep summing it up
        if n == 0:
            raise ValueError("City does not experience this season") #if it
never occurs in the dataset
        else:
            sea_av['average_temperature'] /= n #average -> divide sum by number
of occurences

    return sea_av
```

3. compare_decades() - Perform climate change analysis by comparing temperature averages between decades

```
def compare_decades(filename, city_name, decade1, decade2):
    """
    Compare average temperatures between two decades for a city.

    Parameters:
    filename (str): Path to the CSV file
    city_name (str): Name of the city
    decade1 (int): First decade (e.g., 1980 for 1980s)
    decade2 (int): Second decade (e.g., 2000 for 2000s)

    Returns:
    dict: {
```

```
        'city': str,
        'decade1': {'period': '1980s', 'avg_temp': float, 'data_points': int},
        'decade2': {'period': '2000s', 'avg_temp': float, 'data_points': int},
        'difference': float,
        'trend': str  # 'warming', 'cooling', or 'stable'
    }

    """
    if city_name not in get_available_cities(filename):
        raise ValueError("City not found")

    dec_av = {
        'city': city_name,
        'decade1': {'period': str(decade1)+'s', 'avg_temp': 0, 'data_points':
0},
        'decade2': {'period': str(decade2)+'s', 'avg_temp': 0, 'data_points':
0},
        'difference': 0,
        'trend': ''  # 'warming', 'cooling', or 'stable'
    }


    year1 = decade1
    year2 = decade2

    if year1 % 10 != 0 or year2 % 10 != 0: #in case it isn't a proper decade.
        raise ValueError("Invalid Decade, should end with 0")

    with open(filename, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file)

        for row in reader:
            if row['City'] == city_name:

                 if row['AverageTemperature'] == '':
                    continue
                year = int(row['dt'][:4])

        If the year is in the range of decade 1, add it to the first sum.
                if year1 <= year < year1+10: #if it is in the range of decade 1
                    dec_av['decade1']['avg_temp'] +=
float(row['AverageTemperature'])
                    dec_av['decade1']['data_points'] += 1

                if year2 <= year < year2+10: #if it is in the range of decade 2
                    dec_av['decade2']['avg_temp'] +=
float(row['AverageTemperature'])
                    dec_av['decade2']['data_points'] += 1
```

Raises error if insufficient data.
(no datapoints)

```
        if dec_av['decade1']['data_points'] == 0 or
dec_av['decade2']['data_points'] == 0:
            #insufficient data in one decade
            raise ValueError("Insufficient data")
        else:
            dec_av['decade1']['avg_temp'] /= dec_av['decade1']['data_points']
            dec_av['decade2']['avg_temp'] /= dec_av['decade2']['data_points']

            difference = dec_av['decade1']['avg_temp'] -
dec_av['decade2']['avg_temp']
            dec_av['difference'] = abs(difference)

            difference *= (year1 - year2) # to make sure we capture the trend
forward in time
            if difference > 0:
                dec_av['trend'] = 'warming'
            elif difference == 0:
                dec_av['trend'] = 'stable'
            else:
                dec_av['trend'] = 'cooling'

    return dec_av
```

4. find_similar_cities() - Find cities with similar climate profiles based on temperature tolerance

Created a new function to extract average temps of all cities, as using the first function repeatedly for all cities would be opening and closing the file too many times. It can be more efficiently extracted in just one iteration.

```
# function to extract all cities avg temps in one opening of file
# as the data for one city is in consecutive rows
def all_avg_temps(filename):

    city_to_tc = {} # city_name:(avg temp, country)

    with open(filename, 'r', encoding='utf-8') as file:
        reader = csv.DictReader(file)
        n = 0
        first_iter=1
        citysum=0
        for row in reader:
            temp=row['AverageTemperature']
            if temp=='':
                continue

            curr_city=row['City']
```

```
            if first_iter: #ignore first iteration
                first_iter=0
                n=1
                prev_city=curr_city
                prev_country=row['Country']
                citysum=float(row['AverageTemperature'])
                continue
```

One city's data is in continuous rows, which is why we can run the code to go through continuous lines for a city. If this city is different from the city in the previous iteration, we add the previous city into the dict.

```
            if prev_city==curr_city: #add to the running sum for the city
                n+=1
                citysum+=float(row['AverageTemperature'])

            else: # add the average to the dictionary
                city_to_tc[prev_city]= (citysum/n ,prev_country)
                prev_country=row['Country']
                prev_city=row['City']
                n=1
                citysum=float(row['AverageTemperature'])

        city_to_tc[prev_city]= (citysum/n ,prev_country)
    return city_to_tc



def find_similar_cities(filename, target_city, tolerance=2.0):
    """
    Find cities with similar average temperatures to the target city.
    """
    sim_cities = {
        'target_city': target_city,
        'target_avg_temp': 0,
        'similar_cities': [],
        'tolerance': tolerance
    }

    if target_city not in get_available_cities(filename):
        raise ValueError("City not found")

    mydict = all_avg_temps(filename)
    temp_mid = mydict[target_city][0]
    th = temp_mid + tolerance
    tl = temp_mid - tolerance

    # the range for the temps is [tl, th]
```

```
    for city, tc in mydict.items():
        av_temp = tc[0]
        #check if city is not target city, and check if it falls in the range
        if city != target_city and (tl <= av_temp <= th):
            sim_cities['similar_cities'].append({
                'city': city,
                'country': tc[1],
                'avg_temp': av_temp,
                'difference': abs(av_temp-temp_mid)
            })
    return sim_cities
```

5. get_temperature_trends() - Analyze long-term temperature trends using moving averages and identify warming/cooling periods

```
def get_temperature_trends(filename, city_name, window_size=5):
    """
    Calculate temperature trends using moving averages and identify patterns.
    """
    trends={
        'city': city_name,
        'raw_annual_data': {}, #'YYYY': float
        # Annual averages
        'moving_averages': {},  #'YYYY': float
        # Moving averages
        'trend_analysis': {
            'overall_slope': 0,  # °C per year
            'warming_periods': [], #{'start': year, 'end': year, 'rate': 0}
            'cooling_periods': [] #{'start': year, 'end': year, 'rate': 0}
        }
        }

    if city_name not in get_available_cities(filename):
        raise ValueError("City not found")

    monthwise = get_city_temperatures(filename, city_name)
    prev_year = next(iter(monthwise))[:4] #first year in the dataset

    y_sum = 0
    n = 0
    #calculate annual averages
    for date, temp in monthwise.items():
        year = date[:4]
```

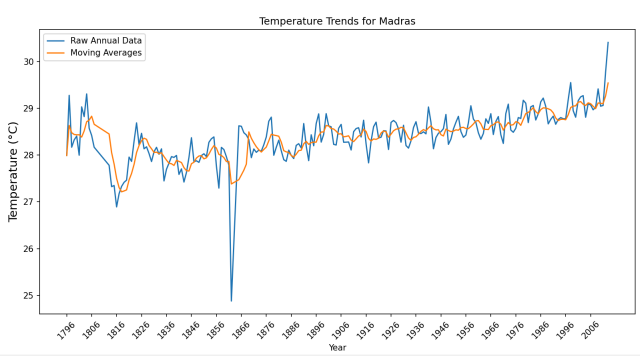
Every time we encounter a new year, save previous year’s data, and initialise data for the new year.

```
        if year != prev_year:
            trends['raw_annual_data'][prev_year] = y_sum/n
            y_sum = temp
```

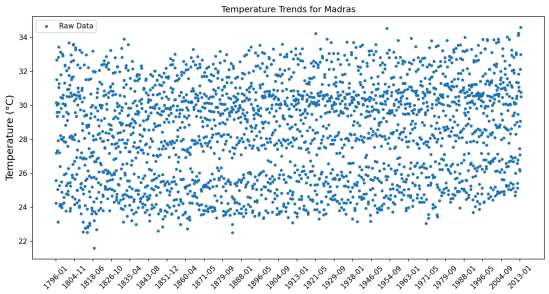
```
            prev_year = year
            n = 1
        else:
Keep the running sum going otherwise.
            y_sum += temp
            n += 1
    trends['raw_annual_data'][prev_year] = y_sum/n

    #calculate moving averages
    list_of_years = []
    list_of_temps = []
    for year, temp in trends['raw_annual_data'].items():
        list_of_years.append(year)
        list_of_temps.append(temp)

    mvavg = []
    m_sum = 0
    #first window_size elements (less than window size if not enough data)
    for i in range(min(window_size, len(list_of_years))):
        m_sum += list_of_temps[i]
        mvavg.append(m_sum/(i+1))

    #sliding window, we delete what was last added, and add what is new
    for i in range(len(list_of_years)-window_size): #i is the index of the num
we delete
        m_sum = m_sum+list_of_temps[i+window_size]-list_of_temps[i]
        mvavg.append(m_sum/window_size)

    #construct the dictionary for moving averages
    for i in range(len(list_of_years)):
        trends['moving_averages'][list_of_years[i]] = mvavg[i]

    #trend analysis
    l=len(list_of_years)
    if l<2:
        raise ValueError("No Trend")


Overall slope = (final - initial) / total observations
    trends['trend_analysis']['overall_slope'] =
(list_of_temps[-1]-list_of_temps[0]) /
(int(list_of_years[-1])-int(list_of_years[0]))


    flag = list_of_temps[1]-list_of_temps[0] # warming positive, cooling
negative
    start_ind = 0 start_ind = start year of the trend

    for i in range(1, len(list_of_temps)):
```

```
        if (list_of_temps[i]-list_of_temps[i-1])*flag >= 0: # this means they
are going in the same trend
            continue
        #if not,
If a trend is broken, we save the previous trend in the dictionary.
        if flag > 0:
            trends['trend_analysis']['warming_periods'].append({
                'start': list_of_years[start_ind],
                'end': list_of_years[i-1],
                'rate':
(list_of_temps[start_ind]-list_of_temps[i-1])/(int(list_of_years[start_ind])-in
t(list_of_years[i-1]))
            })

        else:
            trends['trend_analysis']['cooling_periods'].append({
                'start': list_of_years[start_ind],
                'end': list_of_years[i-1],
                'rate':
(list_of_temps[start_ind]-list_of_temps[i-1])/(int(list_of_years[start_ind])-in
t(list_of_years[i-1]))
            })

        flag = list_of_temps[i]-list_of_temps[i-1]
        start_ind = i-1 #if say, next datapoint changes trend, the changed
trend starts from the previous datapoint.

    return trends
```

Trends for Madras



get_city_temperatures() for Madras.


Temperature Trends for Madras
Raw Data
Temperature (°C)
34
32
30
28
26
24
22
1796-01
1804-11
1818-06
1826-10
1833-04
1843-08
1851-12
1860-04
1871-05
1879-09
1888-01
1896-05
1904-09
1913-01
1921-05
1929-09
1938-01
1946-05
1954-09
1963-01
1971-05
1979-09
1988-01
1996-05
2004-09
2013-01